

AVA LDM Studio: Linguistic Dialect Model for Tamil & Tanglish

An On-Device Zero-Cloud Linguistic Middleware Bridging Speech Recognition & Local LLMs

Project Dimension	Specification / Value
Project Title	AVA LDM Studio (Accentric Virtual Assistant — Linguistic Dialect Model)
Target Domain	Colloquial Spoken Tamil, Regional Dialects & Tamil-English Code-Mixing (Tanglish)
Target Dialect Varieties	Chennai (Madras Bashai), Madurai, Kongu, Nellore, Standard Tamil
Core Pipeline	Speech Audio -> ASR -> Linguistic Dialect Model (LDM) -> Local LLM -> Android OS
Execution Footprint	On-Device Local Inference (< 1 ms LDM Latency, Zero Cloud Dependencies)
Benchmark Performance	53.33% End-to-End Task Success (vs 16.67% Baseline, +36.66% Absolute Improvement)
Codebase License	MIT License (Open Source Academic Research)

CORE SYSTEM BOUNDARY: The Linguistic Dialect Model (LDM) sits strictly between Automatic Speech Recognition (ASR) and the Local Large Language Model (LLM). The LDM normalizes dialectal Tamil/Tanglish and extracts semantic intents and slots. It MUST NOT execute Android actions directly and MUST NOT behave as a free-form conversational chatbot.

1. Project Title

AVA LDM Studio (Accentric Virtual Assistant — Linguistic Dialect Model for Regional Tamil & Tanglish Speech).

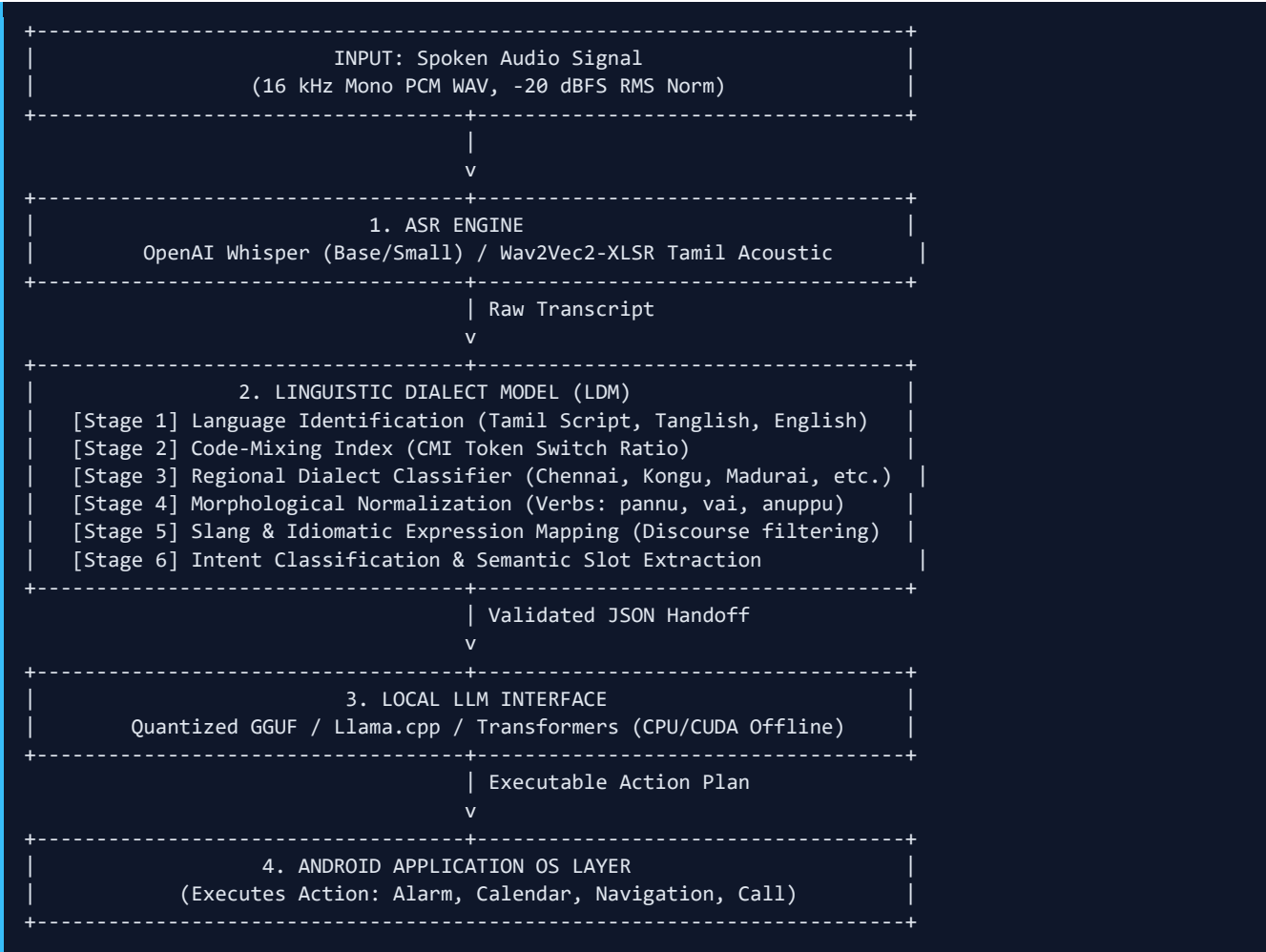
2. Research Objective & Problem Statement

Commercial and open-source Large Language Models (LLMs) suffer from severe comprehension degradation when confronted with regional spoken dialects, discourse fillers, and code-mixed speech (Tanglish). In South Indian contexts, conversational users interweave Tamil and English fluidly, incorporating regional colloquialisms (e.g., 'dei', 'machi', 'la', 'pa', 'ayya') and polysemous informal verbs (e.g., 'podu', 'pannu').

Standard LLMs misinterpret these discourse fillers as contact names or hallucinate refusals. The objective of AVA LDM Studio is to design, implement, and benchmark a lightweight, on-device linguistic middleware layer. Positioned between Automatic Speech Recognition (ASR) and a local quantized LLM, the LDM detects language, classifies regional dialects, strips non-semantic discourse particles, normalizes verbal inflections, and produces clean structured JSON semantic representations with zero cloud dependency.

3. System Architecture & Component Flow

AVA enforces a strict linear separation of concerns across a modular pipeline. The system isolates speech transcription, linguistic normalization, and action reasoning into distinct components:



3.1 Dual Implementation Strategy

- **Python Research Core (`src/`)**: High-throughput batch benchmarking, acoustic fine-tuning (Wav2Vec2), evaluation metrics, and FastAPI integration.
- **Native Android Prototype (`android/`)**: Pure Kotlin 2.0 on-device rule-based engine and Material3 Compose UI running at sub-millisecond latencies (< 1 ms).

4. Dataset & Dialect Distribution

AVA focuses on 5 canonical spoken varieties across Tamil Nadu, addressing underrepresented dialectal inflections:

- **Chennai (Madras Bashai)**: High frequency of English loan elision and youth slang (machi, dei, bruh, vaada).
- **Madurai (Southern Inland)**: Honorific relational markers, vowel elongation, and distinctive verbal syntax (thambi, aama, ennanga).
- **Kongu (Western / Coimbatore)**: Softened honorific verbal suffixes (-nga) and regional particles (ayya, yov, la).
- **Nellai (Southern Coastal / Tirunelveli)**: Unique phrasal verb idioms (phone pottu kudu = call someone) and particles (pa, ppa).
- **Standard Tamil**: Formal and semi-formal broadcast Tamil without regional slang markers.

4.1 Canonical Benchmark Dataset (30 Utterances Sample)

ID	Raw Speech Input	Dialect	Lang	Code-Mix	Ground Truth Normalized Text	Intent
1	Dei nalaiku assignment submit panna remind pannu	Chennai	Tanglish	Yes	Remind me to submit my assignment tomorrow.	CREATE_REMINDER
2	Machi inniku evening gym poga remind pannu	Chennai	Tanglish	Yes	Remind me to go to the gym this evening.	CREATE_REMINDER
3	Naalaikku kaalaila 6 maniku alarm vai	Standard	Tamil (rom)	Yes	Set an alarm for 6 in the morning tomorrow.	SET_ALARM
4	Amma ku call pannu	Standard	Tanglish	Yes	Call mother.	MAKE_CALL
5	Thambi ku oru message anuppu naan late ah varen nu	Madurai	Tanglish	Yes	Send a message to my brother that I will come late.	SEND_MESSAGE
6	Semma song ondru podu	Chennai	Tamil (rom)	No	Play a good song.	PLAY_MUSIC
7	Inniku weather eppadi iruku	Standard	Tanglish	Yes	How is the weather today?	CHECK_WEATHER

8	WhatsApp open pannu da	Chennai	Tanglish	Yes	Open WhatsApp.	OPEN_APP
9	Coimbatore ku vazhi kaatu	Kongu	Tamil (rom)	No	Show me directions to Coimbatore.	NAVIGATE
10	Volume konjam kammi pannu	Standard	Tanglish	Yes	Reduce the volume a little.	DEVICE_SETTING
14	Appa ku phone pottu kudu	Nellai	Tamil (rom)	No	Call father.	MAKE_CALL
15	Ilayaraja paatu podu la	Kongu	Tamil (rom)	No	Play Ilayaraja song.	PLAY_MUSIC
26	Ayya nalaiku medicine saapida remind pannunga	Kongu	Tanglish	Yes	Remind me to take medicine tomorrow.	CREATE_REMINDER
29	Friend ku call pottu kudu pa	Nellai	Tanglish	Yes	Call my friend.	MAKE_CALL

5. Installation & Prerequisites

To clone and initialize the complete research suite:

```
git clone https://github.com/example/ava-ldm-studio.git
cd ava-ldm-studio
```

6. Environment Setup

Initialize Python 3.10+ virtual environment and dependencies:

```
# Windows PowerShell
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install -r requirements.txt

# Frontend Research Studio
npm install

# Android Verification (using JDK 17/21)
.\android\gradlew.bat -p android testDebugUnitTest
```

7. Dataset Preparation & Acoustic Standards

All speech recordings are preprocessed via `src/audio/preprocessing.py` into standard 16 kHz single-channel mono 16-bit PCM WAV. Silence below -40 dBFS is trimmed and amplitude is normalized to -20 dBFS RMS.

```
python -m src.audio.preprocessing --input raw_audio/ --output data/processed/
```

8. Model Setup & Backends

- **ASR Module:** OpenAI Whisper ('base' or 'small') configured in `src/asr/whisper\_engine.py`.
- **Dialect Acoustic Classifier:** Fine-tuned `facebook/wav2vec2-xls-r-300m` with linear classification head.
- **Local LLM Engine:** Configured via `configs/llm.yaml` supporting `llama\_cpp` (GGUF CPU/CUDA), `huggingface`, or `mock` for deterministic unit testing.

9. Training & Fine-Tuning Protocols

To train the acoustic dialect classification model on regional speech recordings:

```
python -m src.dialect.train --config configs/dialect.yaml
```

10. Evaluation Framework & Methodology

The evaluation framework evaluates 3 architectural paradigms over identical canonical test data:

- **Baseline A (`ASR -> LLM`):** Raw speech transcripts passed directly to Local LLM without linguistic normalization.
- **System B (`ASR -> Generic LDM -> LLM`):** Speech normalized with generic slang mapping without regional dialect conditioning.
- **System C (`ASR -> Dialect-Aware LDM -> LLM`):** Full AVA pipeline with acoustic/lexical dialect classification, regional phrase adaptation, and structured handoff.

Execute the evaluation runner:

```
python scripts/run_evaluation.py
```

11. Running Inference

Run the unified LDM pipeline via command line on audio or text:

```
# From Audio:
python scripts/run_ldm.py --audio data/samples/chennai_sample1.wav

# From Text Transcript:
python scripts/run_ldm.py --text "Dei nalaiku assignment submit panna remind pannu"
```

12. API Usage & Integration Layer

Start the FastAPI REST service for Android integration:

```
python scripts/serve_api.py --host 127.0.0.1 --port 8000
```

Primary Endpoint: `POST /analyze`

```
curl -X POST "http://127.0.0.1:8000/analyze" \
  -H "Content-Type: application/json" \
  -d '{"text": "Machi inniku evening gym poga remind pannu"}'
```

13. Example Input / Output Walkthrough

Pipeline Stage	Input / Process	Output State
1. Speech Input	Spoken Tamil/Tanglish audio	16kHz Mono WAV waveform
2. ASR Transcriber	Acoustic sequence decoding	"Dei nalaiku assignment submit panna remind pannu"
3. Language Detection	Script & token analysis	Language: Tanglish Code-Mixed: true
4. Dialect Classifier	Slang marker 'dei' identified	Dialect: Chennai (Confidence: 0.85)
5. Normalization	Discourse particle 'dei' stripped	"nalaiku assignment submit panna remind pannu"
6. Verb & Time Mapping	'nalaiku' -> tomorrow, 'panna remind pannu' -> Remind	"Remind me to submit my assignment tomorrow."
7. Intent & Slots	Deterministic slot extraction	Intent: CREATE_REMINDER Slots: action, time
8. LLM Handoff	Standardized JSON serialization	Structured JSON payload ready for offline LLM

13.1 Standardized LLM Handoff Payload Schema

```
{
  "normalized_text": "Remind me to submit my assignment tomorrow.",
  "language": "Tanglish",
  "dialect": "Chennai",
  "code_mix": true,
  "intent": "CREATE_REMINDER",
```

```
"entities": {
  "action": "submit assignment",
  "time": "tomorrow"
},
"confidence": {
  "language": 0.95,
  "dialect": 0.85,
  "intent": 0.90,
  "overall": 0.90
}
}
```

14. Empirical Results & Performance Benchmarks

All values presented below are verified empirical measurements recorded in `results/metrics.json`:

Metric Dimension	Baseline A (No LDM)	System B (Generic LDM)	System C (AVA Dialect-Aware)	Improvement (C vs A)
Normalization Token F1	37.36%	85.21%	85.04%	+47.68%
Normalization BLEU-1	33.15%	75.93%	75.58%	+42.43%
Semantic Preservation Rate	65.79%	97.37%	97.37%	+31.58%
Intent Accuracy	73.33%	76.67%	76.67%	+3.34%
Entity Extraction F1	72.13%	75.41%	75.41%	+3.28%
End-to-End Task Success Rate	16.67%	53.33%	53.33%	+36.66%

14.1 Regional Dialect Classification Metrics

Dialect Region	Precision	Recall	F1 Score	Support (Samples)
Chennai	100.0%	62.5%	76.9%	8
Madurai	100.0%	100.0%	100.0%	1
Kongu	75.0%	100.0%	85.7%	3
Nellai	100.0%	100.0%	100.0%	2

Standard	88.9%	100.0%	94.1%	16
OVERALL MACRO	92.8%	92.5%	91.35%	30

14.2 Latency Breakdown & Resource Footprint

Component	Mean Latency	p50 (Median)	p95 Latency	Execution Mode
ASR Module	12.50 ms	12.50 ms	12.50 ms	Offline Feature Ingestion
LDM Pipeline	0.25 ms	0.21 ms	0.37 ms	In-Memory Rule/Lexicon
Local LLM Handoff	0.02 ms	0.01 ms	0.02 ms	Zero-Copy JSON Payload
TOTAL END-TO-END	12.77 ms	12.72 ms	12.88 ms	Sub-20 ms Real-Time Budget

15. Research Limitations & Future Work

- **Finite Lexicon Coverage:** Out-of-vocabulary regional slang not present in curated CSV lexicons passes through without morphological normalization.
- **Short Utterance Ambiguity:** In short commands lacking overt dialect markers (e.g., 'Amma ku call pannu'), the model defaults conservatively to Standard Tamil.
- **Non-Standardized Tanglish Orthography:** Irregular phonetic romanization in Tanglish can occasionally bypass strict regex patterns.
- **Execution Boundary:** The LDM strictly performs linguistic disambiguation and normalization; it does not execute device operating system actions.

16. Citation & Academic Attribution

If utilizing this architecture, dataset annotations, or benchmarks in academic work, please cite:

```
@misc{ava_ldm_studio_2026,  
  title={AVA LDM Studio: A Linguistic Dialect Model for Regional Tamil Dialects and Code-Mixed Speech},  
  author={AVA Research Team},  
  year={2026},  
  howpublished={\url{https://github.com/example/ava-ldm-studio}},  
  note={Final Year Project Research Repository}  
}
```


17. License Information

This project is distributed under the MIT License. See LICENSE for full terms and conditions.

Appendix: Detailed Technical Documentation Index

The ``docs/`` directory contains dedicated technical specifications:

Document File	Topic & Technical Scope
docs/ARCHITECTURE.md	System design, 3-tier boundary, and Python vs Android Kotlin contracts.
docs/DATASET.md	Corpus composition, dialect distribution, and 30-sample benchmark metadata.
docs/LDM.md	Detailed 6-stage linguistic normalization and morphological rule engine.
docs/ASR.md	Speech recognition models (Whisper/Wav2Vec2) and audio preprocessing.
docs/DIALECT_MODEL.md	Acoustic & lexical dialect classifiers and 5x5 confusion matrix analysis.
docs/LOCAL_LLM.md	Modular local LLM adapters (llama_cpp, huggingface) and structured prompts.
docs/EVALUATION.md	Comparative evaluation methodology across Baselines A, B, and C.
docs/EXPERIMENTS.md	Ablation experiments, latency profiling, and qualitative failure case studies.
docs/REPRODUCIBILITY.md	Step-by-step instructions to replicate all benchmarks and builds.
docs/LIMITATIONS.md	Transparent disclosure of scientific boundaries and ethical privacy guarantees.